In [10]:
```
#LOGIN SYSTEM (auth.py)
#Stores Username and Password
#Password is converted into hash

import sqlite3
import hashlib

def init_db():
    conn = sqlite3.connect("users.db")
    cur = conn.cursor()
    cur.execute("CREATE TABLE IF NOT EXISTS users (username TEXT, password TEXT)")
    conn.commit()
    conn.close()

def register(username, password):
    conn = sqlite3.connect("users.db")
    cur = conn.cursor()
    hashed = hashlib.sha256(password.encode()).hexdigest()
    cur.execute("INSERT INTO users VALUES (?, ?)", (username, hashed))
    conn.commit()
    conn.close()

def login(username, password):
    conn = sqlite3.connect("users.db")
    cur = conn.cursor()
    hashed = hashlib.sha256(password.encode()).hexdigest()
    cur.execute("SELECT * FROM users WHERE username=? AND password=?", (username, h
    result = cur.fetchone()
    conn.close()
    return result is not None
```

In [ ]:
```
#AES IMAGE ENCRYPTION(aes.py)
#Converts image into bytes
#Encrypts using AES
#Saves encrypted File

from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes

def encrypt_image(input_file, output_file):
    key = get_random_bytes(16)
    cipher = AES.new(key, AES.MODE_EAX)

    with open(input_file, 'rb') as f:
        data = f.read()

    ciphertext, tag = cipher.encrypt_and_digest(data)

    with open(output_file, 'wb') as f:
        f.write(cipher.nonce + tag + ciphertext)

    return key

def decrypt_image(input_file, key, output_file):
```

```
    with open(input_file, 'rb') as f:
        nonce = f.read(16)
        tag = f.read(16)
        ciphertext = f.read()

    cipher = AES.new(key, AES.MODE_EAX, nonce)
    data = cipher.decrypt_and_verify(ciphertext, tag)

    with open(output_file, 'wb') as f:
        f.write(data)
```

In [4]:

```
#RSA KEY SECURITY (rsa.py)
#AES key is protected using RSA
#Only server can open it

from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP

def generate_keys():
    key = RSA.generate(2048)
    return key.export_key(), key.publickey().export_key()

def encrypt_key(aes_key, public_key):
    rsa_key = RSA.import_key(public_key)
    cipher = PKCS1_OAEP.new(rsa_key)
    return cipher.encrypt(aes_key)

def decrypt_key(enc_key, private_key):
    rsa_key = RSA.import_key(private_key)
    cipher = PKCS1_OAEP.new(rsa_key)
    return cipher.decrypt(enc_key)
```

In [ ]:

```
#SERVER(server.py)
#Recieves encrypted file + key

from flask import Flask, request
import os

app = Flask(__name__)
os.makedirs("received", exist_ok=True)

@app.route('/upload', methods=['POST'])
def upload():
    file = request.files['file']
    key = request.files['key']

    file.save("received/encrypted.enc")
    key.save("received/key.enc")

    return "Received securely"

app.run(debug=True)
```

In [6]:

```
#CLIENT(client.py)
#Sends encrypted data to server
```

```python
import requests

def send_file(file_path, key_path):
    url = "http://127.0.0.1:5000/upload"

    files = {
        'file': open(file_path, 'rb'),
        'key': open(key_path, 'rb')
    }

    response = requests.post(url, files=files)
    print(response.text)
```

In [ ]:

```python
#MAIN PROGRAM(main.py)
#Combines everything

from auth import init_db, register, login
from aes import encrypt_image
from rsa import generate_keys, encrypt_key
from client import send_file

init_db()

print("1. Register\n2. Login")
choice = input("Enter choice: ")

username = input("Username: ")
password = input("Password: ")

if choice == "1":
    register(username, password)
    print("Registered Successfully")

elif choice == "2":
    if login(username, password):
        print("Login Successful")

        # Generate RSA keys
        private_key, public_key = generate_keys()

        # Encrypt image
        aes_key = encrypt_image("input.jpg", "encrypted.enc")

        # Encrypt AES key
        enc_key = encrypt_key(aes_key, public_key)

        with open("key.enc", "wb") as f:
            f.write(enc_key)

        # Send file
        send_file("encrypted.enc", "key.enc")

    else:
        print("Invalid Login")
```